AI 서비스 Mini-Project · 과제 1번

# 운영 인시던트 대응 에이전트 개선하기

**원본을 같은 질문 9개로 먼저 재고, 찾은 문제를 고쳤습니다**

G102 · 최은주

# Contents


01 진행 사항 — 문제와 수정·추가 task

02 기술 요소와 구현 화면

03 개선 전/후 수치

04 코드 설명 — Graph · MCP · RAG

05 한계와 이후 개선

# 먼저 원본을 쟀습니다

## PDF 8쪽 질문 9개 + 정답 라벨로 자동 채점

| 영역 | 원본에서 실제로 본 문제 | 근거 (Before 측정) |
|---|---|---|
| 속도 | qwen3 thinking이 기본으로 켜짐, 동기 LLM 호출이 서버를 멈춤 | 장애 6건 269초 · 처리 중 /health 지연 |
| 정확도 | 시스템 2개를 말해도 1개만 조사, billing-service를 auth-service로 바꿔치기 | 누락 3건 · 미등록 안내 0/1 |
| runbook | 키워드 규칙이 healthy 시스템이면 db-down으로 폴백 | runbook 4/6 · 규칙 77% |
| 검증 | validator가 “30자 이상 + 1) 포함”만 확인, 근거 검사 없음 | 조사 안 한 시스템을 언급한 답도 통과 |
| 운영·구조 | 위험 조치를 확인 없이 안내, 긴 대화에서 이름을 잊음, graph.py 한 파일 338줄 | 승인 단계 없음 · 최근 10개 메시지만 전달 |

# 기존 기능 개선 7개 + 구조 개선·기능 추가 8개

## [개선] 원래 있던 기능

- 1 성능: thinking 끄기 + 동기 호출을 to_thread로
- 2 정확도: 조사 대상은 쿼리가 결정, 미등록은 안내
- 3 RAG: 키워드 규칙 → runbook 검색
- 4 검증: 답변 근거 검사 후 재작성
- 5 운영: 실행 스크립트 안전화, 화면 버그
- 6 테스트: pytest (LLM·MCP 없이 실행)
- 7 코드 리뷰로 찾은 버그 4건

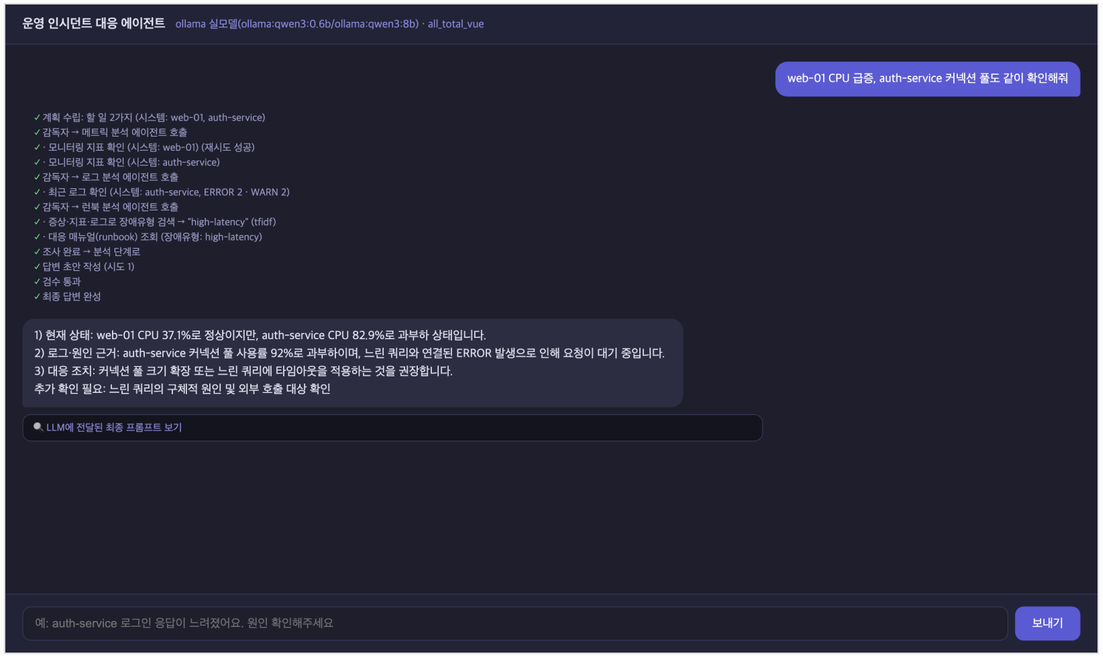
## [추가] 새로 넣은 기능

- 8 모듈화: 노드를 agent/nodes/로 분리
- 9 되묻기: 대상이 불명확하면 clarify
- 10 멀티 에이전트: 메트릭·로그·런북 서브그래프
- 11 Human-in-the-loop 승인 노드
- 12 대화 요약 메모리
- 13 LLM-as-judge 평가자
- 14 리뷰 반영 · 15 OpenAI 다듬기 + LangSmith

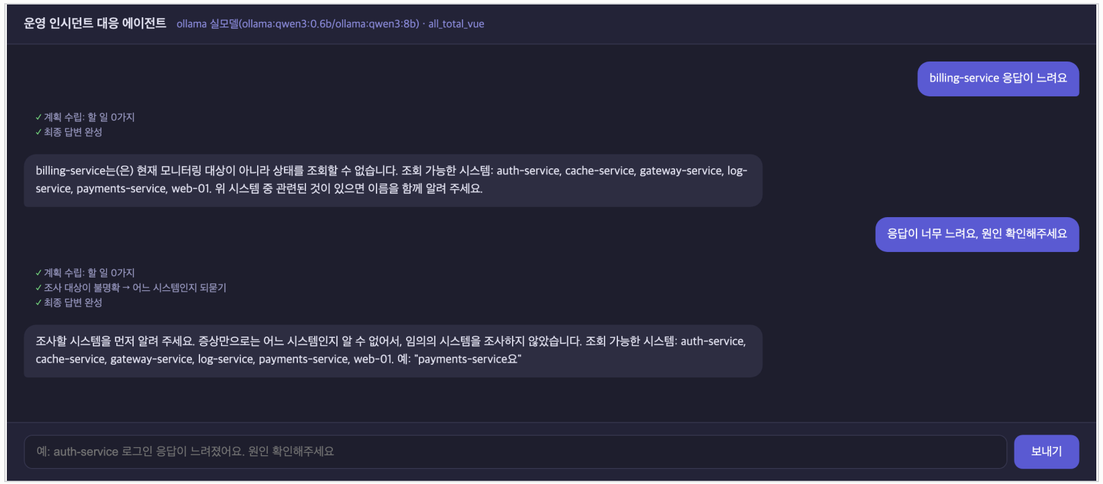
# 평가표의 기술 요소를 어디에 썼나

| 기술 요소 | 이 과제에서 한 것 | 파일 |
|---|---|---|
| Multi-Agent (구조) | 감독 노드 + 메트릭·로그·런북 compiled subgraph 3개 (안쪽은 LLM 없이 도구 조회, 순서는 코드) | agent/research/ |
| MCP | 로그 조회 Tool(get_recent_logs) 추가, 원본의 runbook Resource 조회를 런북 서브그래프로 옮김 | mcp_server.py |
| RAG | runbook 줄 단위 조각 + incident_type 메타데이터 TF-IDF 검색 | agent/runbook_retriever.py |
| 의존성 주입·모듈화 | 노드는 Deps를 partial로 주입받는 함수, graph.py는 엣지만 | agent/nodes/, graph.py |
| LangSmith | 노드에 @traceable, 본인 키로 추적을 켜 노드별 시간 측정 (10장) | eval/langsmith_breakdown.py |
| 단위 테스트 | pytest 80개, LLM·MCP 없이 3.6초 | tests/ |
| HITL · 메모리 · 평가 | interrupt() 승인, 요약 메모리, LLM-as-judge | nodes/approval.py 외 |

# 말한 시스템은 전부 조사하고, 모르면 찍지 않는다

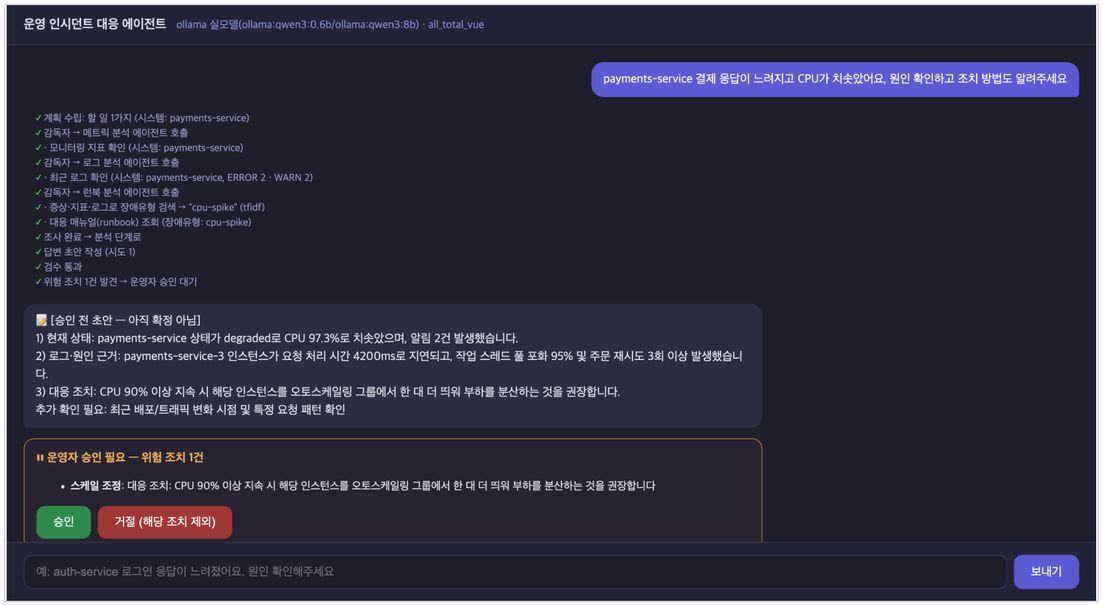



시스템 2개 → **둘 다 조사**
저하된 auth-service만 로그 확인 · runbook은 high-latency

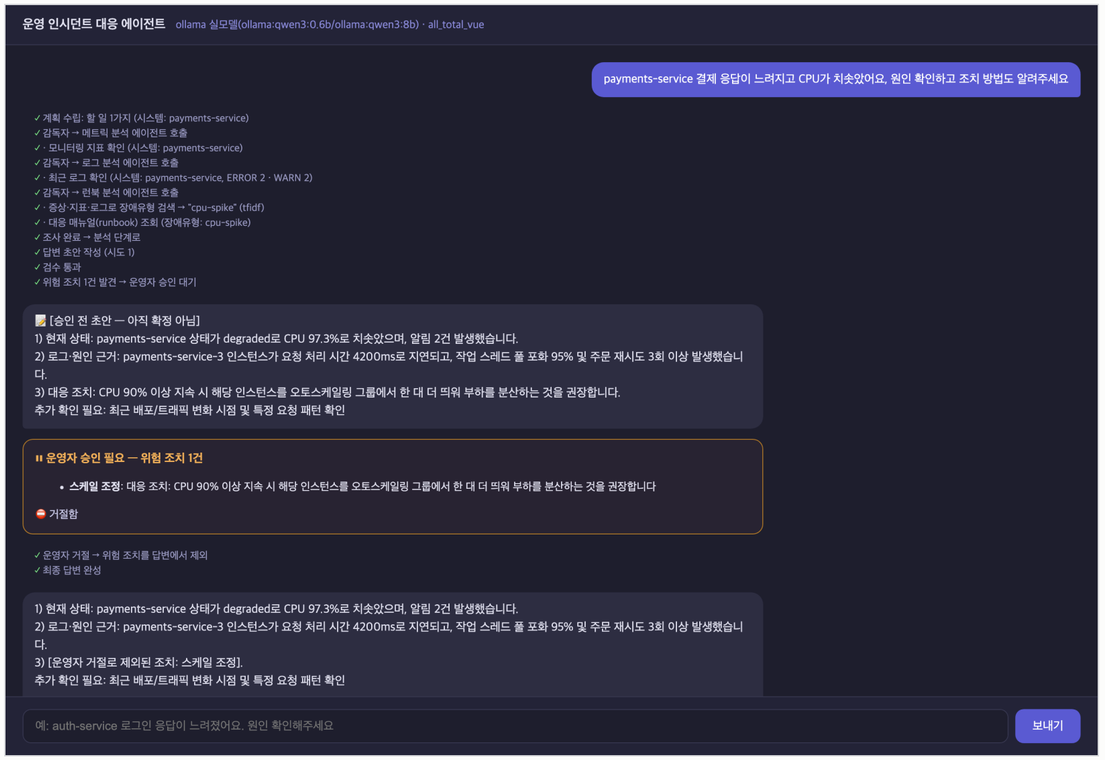



미등록 시스템 → **조회 가능한 목록 안내**(0.2초)
시스템 없이 증상만 → **되묻기**, 다음 턴에 증상과 합쳐 재개

# 위험 조치는 운영자가 승인해야 답변에 남는다




검수를 통과한 답에 **스케일 조정**이 있어 interrupt()로 멈춤




거절 → 그 조치만 **“운영자 거절로 제외된 조치”**로 바꿔 확정

# 269초 → 51초, 조사 누락 3 → 0

| 지표 | Before | After |
|---|---|---|
| 장애 질문 6개 소요 | 269.4초 | **51.4초** |
| 조사 누락 시스템 | 3 | **0** |
| runbook 정확도 | 4/6 | **6/6** |
| 미등록 시스템 안내 | 0/1 | **1/1** |
| 가상 비용(USD) | 0.0359 | **0.0189 (−47%)** |
| RAG 정확도 (22문항) | 규칙 77% | **95%** |
| 불필요하게 조사한 시스템 | 1 | **0** |

```
$ python3 eval/compare.py before_ollama after_ollama_v2
| 지표 | before_ollama | after_ollama_v2 |
|---|---|---|
| 라우팅 정확도 | 9/9 | 9/9 |
| 장애 쿼리 총 소요(초) | 269.4 | 51.4 |
| 전체 9건 총 소요(초) | 273.5 | 52.0 |
| 조사 누락 시스템 수 | 3 | 0 |
| 불필요하게 조사한 시스템 수 | 1 | 0 |
| runbook 정확도 | 4/6 | 6/6 |
| 미등록 시스템 정직하게 안내 | 0/1 | 1/1 |
| 조사 안 한 시스템·로그를 언급한 답변 수 | 3 | 0 |
| 이름 기억 | True | True |
| 가상 비용 합계(USD) | 0.035891 | 0.018905 |
| 재시도 합계 | 1 | 1 |
| 검증 시도 합계 | 6 | 5 |
| 위험 조치 승인 요청 수(개선 11) | 0 | 2 |


$ python3 eval/rag_eval.py | tail -2
키워드 규칙 정확도: 17/22 = 77%
RAG(tfidf) 정확도: 21/22 = 95%
```

# 요약 메모리는 토큰을 줄이고, 채점기는 먼저 검증했다

## 요약 메모리 — 37턴 대화 끝에 이름·팀 묻기

| 방식 | 기억 | 프롬프트 토큰 |
|---|---|---|
| 최근 10개만 (재현) | 0/2 | 292 |
| 전체 이력 | 1/2 | 1402 |
| **요약 메모리** | **2/2** | **328** |

합성 대화 · '재현'은 원본의 창 10개를 개선 코드로 흉내 낸 것
이름·소속은 정규식으로 고정한 항목이라
2/2는 정규식 효과, LLM 요약 품질은 따로 재지 않음

## LLM-as-judge — 정답 라벨 12건으로 검증

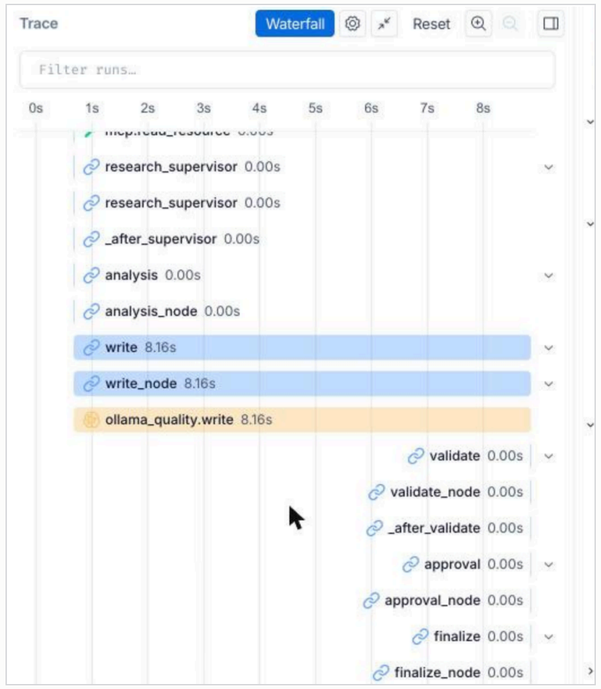
| 채점기 | 8b | 0.6b |
|---|---|---|
| 규칙 검사 (개선 4) | 8/12 | 8/12 |
| 점수형 (1~5점) | 12/12 | 9/12 |
| 주장 확인형 | 12/12 | 10/12 |
| **규칙 + 주장** | **12/12** | **11/12** |

답변당 약 6~11초 → 실시간 검증이 아니라 오프라인 평가에만

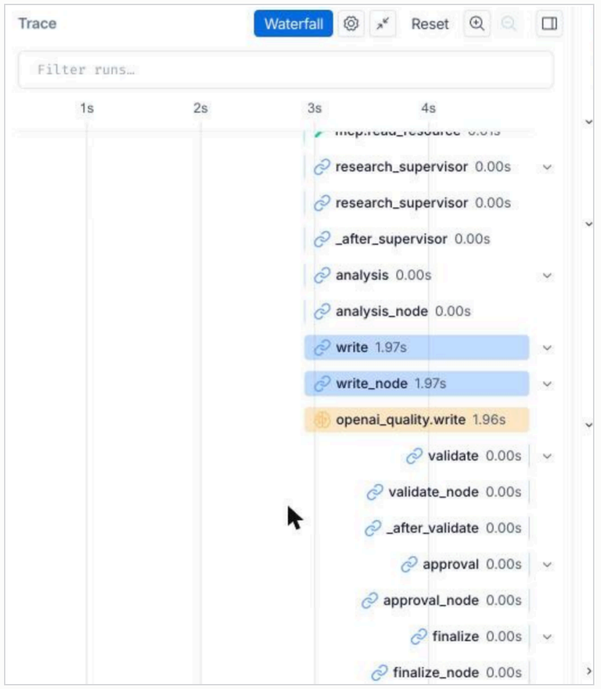
# 백엔드만 바꾸면 정확도는 그대로, 장애 질문은 약 2배 빠르게

| 지표 | 로컬 qwen3 | OpenAI |
|---|---|---|
| 누락 · runbook · 조사 안 한 시스템 언급 | 0 · 6/6 · 0 | 0 · 6/6 · 0 |
| 장애 질문 소요 | 50.0초 | **26.1초** |
| 대화 질문 한 건 | **0.2~0.3초** | 1.7~1.9초 |
| write 노드 비중 (LangSmith) | 86% | **31%** |

다듬기: 형식 위반(덧붙인 문단·빈 “추가 확인 필요”) 4건 → 0건.
1차로 다듬자 “분산하겠습니다”처럼 실행하지 않는 조치를 약속하는 답이 4건 나와 권고형으로 고침

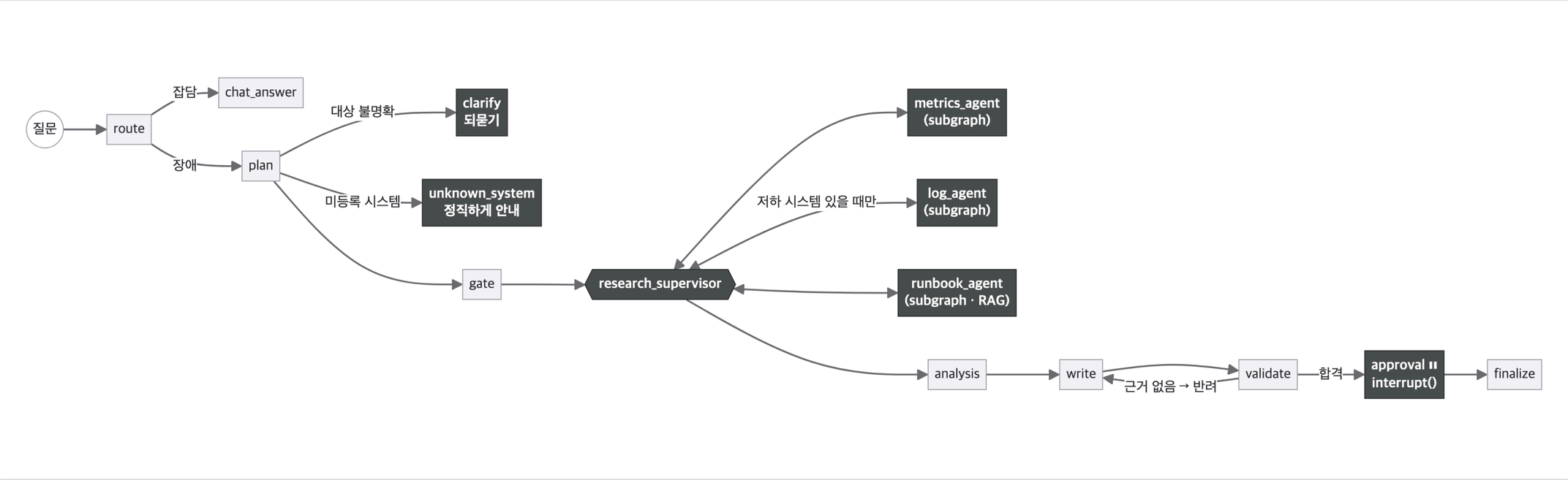



로컬 write 8.16초




OpenAI write 1.97초

# LangGraph — 진하게 표시한 노드가 새로 넣은 것




**plan 뒤 3갈래**
대상 불명확 → clarify, 미등록 → unknown_system, 나머지만 조사

**research_supervisor**
서브에이전트를 하나 부르고 돌아와 다음을 고름

**validate → approval**
검수 합격한 답에만 위험 조치 승인을 물음

# 개선이 새로 만든 약점

| 개선 | 대가 · 한계 |
|---|---|
| thinking 끄기 | 0.6b가 “제 이름은 민수입니다”를 장애로 분류(개발 중 관찰) → 라우팅 게이트로 막았지만 대화 문장은 여전히 어색 |
| 코드가 고르는 감독자 | 무한 루프는 불가능해졌지만, 예상 밖의 조사 순서는 표현하지 못함 |
| TF-IDF RAG | 평가 문항과 메타데이터를 같은 사람이 써서 95%는 낙관적일 수 있음. 임베딩은 설치·지연 비용 |
| HITL | 정규식이라 오탐 감수. 대기 상태가 메모리(InMemorySaver)라 서버 재시작 시 사라짐 |
| 요약 메모리 | 8개 메시지마다 LLM 호출 1번(약 1~1.6초), 요약에서 빠진 세부는 되살릴 수 없음 |
| LLM 채점기 | 필터를 같은 After 답변 5개를 보며 조정해 독립 평가가 아님 · 답변당 약 6~11초라 오프라인에서만 |
| 측정 | 조건별 1회 측정이라 초 단위 값은 실행마다 달라짐 |

# 먼저 재고, 고치고, 다시 잰다

## 측정이 할 일을 정한다

원본을 재 보기 전엔 “느리다” 정도로만 보였다. 9개 질문을 채점하고 나서야 누락, 바꿔치기, runbook 오류가 보였다

## LLM이 안 해도 될 일은 코드로

조사 대상, 다음 에이전트, 로그 개수 세기를 코드로 옮긴 것이 속도와 정확도를 함께 올렸다

## 이후 개선

fast 모델을 qwen3:1.7b로 상향, 임베딩 RAG와 남이 만든 평가셋, SQLite 체크포인터, OpenAI 모드에서 route LLM 호출 줄이기, 측정 3회 평균

# 감사합니다

G102 · 최은주

github.com/5solemi5/base-ai-agent-improved · 보고서 doc/IMPROVEMENTS.md

# graph.py는 연결만, 다음 에이전트는 코드가 고른다

**agent/graph.py — build_graph()**

```
node = lambda fn: partial(fn, deps=deps)  # DI
g.add_node("metrics_agent", partial(
    metrics_agent_node, deps=deps,
    agent=build_metrics_agent(deps.mcp)))
g.add_conditional_edges("research_supervisor",
    _after_supervisor,
    {**{a: a for a in AGENTS}, "analysis": ...})
for a in AGENTS:   # 끝나면 항상 감독자로
    g.add_edge(a, "research_supervisor")
```

**agent/research/supervisor.py — next_agent()**

```
called = state.get("agents_called", [])
if len(called) ⩾ len(AGENTS):  # 하드 상한
    return "done"
if "metrics_agent" not in called:
    return "metrics_agent"
if state.get("degraded") and \
        "log_agent" not in called:
    return "log_agent"   # 저하 시만
if "runbook_agent" not in called:
    return "runbook_agent"
return "done"
```

# interrupt()로 멈추고, Command(resume)로 이어서

**agent/nodes/approval.py — approval_node()**

```
actions = find_risky_actions(state["draft"])
if not actions or not deps.hitl:
    return {...}           # 그대로 통과
# ⚠️ 재개 때 이 위 코드가 한 번 더 돈다
#    → 저장·외부 호출을 두지 않는다
decision = interrupt({"type": "ApprovalRequired",
                      "actions": actions, ...})
draft = apply_decision(state["draft"], actions,
                       decision["approved"], ...)
```

**server.py — POST /chat/approve**

```
pending = _get_pending(req.thread_id)  # 없으면 404
result = await graph.ainvoke(
    Command(resume={"approved": req.approved,
                    "comment": req.comment}),
    config=_run_config(pending["run_key"]))
_finish_run(...)   # 끝난 체크포인트 삭제
```

위험 조치 판정은 정규식(재시작·롤백·스케일·failover·삭제). 놓치지 않는 쪽을 택해 오탐을 감수

# 로그 Tool을 추가하고, 서브에이전트가 MCP로 조회한다

**mcp_server.py — 새 Tool**

```
@mcp.tool()
def get_recent_logs(system_name: str,
        min_level: str = "INFO",
        limit: int = 20) → RecentLogs:
    """최근 로그 조회. min_level=WARN이면
    WARN·ERROR만 돌려줍니다."""
    ...
    return RecentLogs(..., error_count=...,
                      warn_count=...)
```

ops 서버 기준 Tool 2개(모니터링, **로그는 이번에 추가**) · Resource 1(runbook://) · Prompt 1(진단 절차)

**research/log_agent.py · runbook_agent.py**

```
# 저하 시스템들을 병렬로, 실패 시 재시도
await asyncio.gather(*(with_retry(
    mcp.call_tool, "get_recent_logs",
    {"system_name": s, "min_level": "WARN"})
    for s in state["systems"]))
# runbook URI는 하드코딩하지 않는다
server, template = await mcp
    .find_resource_template("get_incident_runbook")
uri = template.format(incident_type=...)
```

반복 ERROR 개수 세기는 LLM 없이 코드(Counter)로

# 키워드 규칙 → runbook 조각 검색 (77% → 95%)

### 원본 graph.py — 키워드 규칙

```
for keyword, itype in _INCIDENT_TYPE_RULES:
    if keyword in text:   # 알림 문구만 봄
        return itype
return "db-down"   # 못 찾으면 무조건
```

### agent/runbook_retriever.py — 인덱스

```
for path in RUNBOOK_DIR.glob("*.md"):  # 6종
    for line in 본문 줄:      # 조각 = 1줄
        chunks.append({"incident_type": itype,
                       "text": line})
    for phrase in meta[itype]:  # "CPU가 치솟음"
        chunks.append({...})
index = _TfidfIndex(조각들)  # 단어 + 문자 2·3-gram
```

### classify() — 무엇으로 검색하나

```
# 사용자 질문 + 로그 ERROR 문구
#  + 저하된 시스템의 알림만 (healthy 제외)
text = query + " " + " ".join(signals)
ranked = self.rank(text)  # 유형별 상위 3조각 합
best, score = ranked[0]
```

| 질문 | 원본 | RAG |
|---|---|---|
| web-01 + auth-service | db-down | **high-latency** |
| payments + cache | cpu-spike | **memory-leak** |